NEXUSNEXUS

Plateforme de Visualisation Trafic en Temps Réel

Documentation Technique Complète

Version 1.0 — Mai 2026

Stack Frontend	Angular · TypeScript · MapLibre GL JS · WebGL2 · RxJS · SCSS
Stack Backend	Spring Boot · Java 21 · WebSocket (STOMP) · Maven
Communication	WebSocket temps réel + REST API
Rendu	WebGL2 instancié — milliers de véhicules en simultané
Carte	OpenStreetMap via Overpass API ou fichier .osm local

Préparé pour présentation académique

Table des Matières

1. Vue d'ensemble du projet
2. Architecture générale
3. Le Backend — Spring Boot
4. Le moteur de simulation
5. Les véhicules — types, vitesse, distance
6. Routage sur réseau routier réel (A*)
7. Modèle de suivi de véhicule (Car-Following)
8. Les feux de circulation
9. Le système d'accidents
10. Gestion du trafic et horloge simulée
11. Les zones géographiques (Geofences)
12. Le Frontend — Angular
13. Rendu WebGL2 — Véhicules
14. Rendu WebGL2 — Trainées de mouvement
15. Rendu WebGL2 — Carte de congestion
16. Événements et incidents en direct
17. Communication WebSocket
18. Tableau de bord et contrôles
19. Filtres et overlays
20. Import de carte OSM
21. Système de replay
22. Performance et optimisations
23. Structure complète des fichiers
24. API REST — Référence complète

1. Vue d'ensemble du projet

NexusNexus est une application web full-stack de **visualisation de trafic en temps réel**. Elle simule des milliers de véhicules se déplaçant sur un réseau routier réel issu d'OpenStreetMap, et les affiche sur une carte interactive avec des effets de rendu GPU avancés.

L'objectif principal est la **performance** : maintenir un framerate fluide avec des milliers de véhicules simultanés, sans jamais bloquer l'interface. Tout le rendu passe par WebGL2, jamais par le DOM. La simulation tourne en Java côté serveur et diffuse l'état via WebSocket toutes les 100ms.

Objectifs du projet

- Afficher et gérer **des milliers de véhicules** en simultané sans perte de fluidité
- Simuler un **comportement réaliste** : suivre de vraies routes OSM, respecter les feux, s'adapter à la densité
- Proposer une **interface professionnelle** : thème sombre, animations fluides, tableau de bord interactif
- Être **scalable** : de 50 à 5000 véhicules sans changer d'architecture
- Permettre l'**import de cartes** : charger n'importe quelle zone OSM à chaud

2. Architecture Générale

Le projet est divisé en deux parties indépendantes qui communiquent via réseau :

Couche	Technologie	Port	Rôle principal
Frontend	Angular + MapLibre GL JS	4200	Affichage carte, WebGL, interface utilisateur
Backend	Spring Boot (Java 21)	8080	Simulation, routage, WebSocket, REST API

Flux de données global

Le backend simule les véhicules à 10 images/seconde (toutes les 100ms) et diffuse leur position via WebSocket. Le frontend reçoit chaque frame, met à jour ses buffers WebGL, et demande un nouveau rendu à MapLibre. Jamais de DOM pour les véhicules — tout est GPU.

Étape	Qui	Quoi
1. Tick simulation	Backend (100ms)	Avance chaque véhicule sur sa route A*
2. Snapshot	Backend	Sérialise position + vitesse + type de chaque véhicule
3. Broadcast WebSocket	Backend → /topic/vehicles	Envoie SimulationFrame JSON
4. Réception	Frontend WebSocketService	Parse et distribue aux renderers
5. Mise à jour GPU	VehicleRenderer (WebGL)	Écrit dans le buffer instancié
6. Rendu	MapLibre (chaque frame)	drawArraysInstanced() — un seul appel GPU

3. Le Backend — Spring Boot

Le backend est un projet **Maven / Java 21** basé sur **Spring Boot 3**. Il démarre sur le port 8080 et expose à la fois une API REST et un endpoint WebSocket (STOMP over SockJS).

Packages Java

Package	Fichier	Rôle
config	SimulationConfig.java	Propriétés YAML (nb véhicules, zone, vitesse...)
config	WebSocketConfig.java	Configure STOMP over SockJS, endpoint /ws
config	CorsConfig.java	Autorise toutes les origines (dev)
model	Vehicle.java	État interne mutable d'un véhicule (position, route, vitesse...)
model	VehicleUpdate.java	Enregistrement immuable broadcast (id, lat, lon, speed, type)
model	VehicleType.java	Enum : CAR / BUS / TRUCK / MOTORCYCLE / EMERGENCY
model	SimulationFrame.java	Enveloppe broadcast : timestamp + nb véhicules + liste updates
model	LightUpdate.java	Position + état (rouge/vert) d'un feu de circulation
model	IncidentUpdate.java	Incident live : type, sévérité, position, expiration
model	ZoneStatus.java	Zone géographique + compte de véhicules dedans
controller	SimulationController.java	API REST /api/simulation/* (pause, vitesse, trafic...)
controller	ReplayController.java	API REST /api/replay/* (liste, lecture)
controller	MapController.java	POST /api/maps/upload — import carte OSM
service	FrameRecorder.java	Enregistre les frames sur disque pour le replay
service	MapImportService.java	Valide + parse un fichier .osm, remplace le graphe
simulation	SimulationEngine.java	Moteur principal — tick, routage, car-following
simulation	VehicleGenerator.java	Crée N véhicules placés sur des arêtes aléatoires
simulation	RoadGraph.java	Graphe immuable : nœuds OSM, adjacence, arêtes, bounds
simulation	RoadNetworkLoader.java	Charge le graphe via l'API Overpass (internet)
simulation	LocalFileRoadGraphProvider.java	Charge le graphe depuis un fichier .osm local
simulation	AStarPathfinder.java	Algorithme A* sur le graphe routier
simulation	TrafficTimeService.java	Horloge simulée 60× temps réel
simulation	TrafficMode.java	Enum : RUSH_HOUR / NORMAL / NIGHT (facteurs de vitesse)
simulation	AccidentSystem.java	Génère des accidents aléatoires (véhicules figés)
simulation	TrafficLightSystem.java	Feux de circulation aux intersections (≥4 voisins)
simulation	ZoneSystem.java	4 zones géographiques + comptage véhicules en temps réel
websocket	VehicleBroadcaster.java	@Scheduled 100ms : tick → record → broadcast WebSocket

4. Le Moteur de Simulation

SimulationEngine.java est le cœur du backend. C'est lui qui fait avancer tous les véhicules à chaque tick, calcule les distances, les feux, les accidents, et prépare le snapshot à diffuser.

Initialisation (au démarrage)

Au démarrage de Spring Boot, @PostConstruct init() lance un **thread de fond** qui :

- Choisit le fournisseur de graphe selon la config (graph-source: overpass ou local-file)
- Parse le graphe OSM (peut prendre plusieurs secondes pour les grandes villes)
- Génère les véhicules initiaux sur des arêtes aléatoires via **VehicleGenerator**
- Construit les feux de circulation via **TrafficLightSystem.build()**
- Pendant ce temps, le serveur REST et WebSocket est déjà disponible (0 véhicule jusqu'au chargement)

Cycle de tick (toutes les 100ms)

Le **VehicleBroadcaster** appelle tick(deltaMs) toutes les 100ms via @Scheduled. Voici ce qui se passe dans l'ordre :

Étape	Méthode	Description
1	trafficTime.advance(deltaMs)	Avance l'horloge simulée (60× temps réel)
2	accidents.update(vehicles, nowMs)	Vérifie si un accident doit se déclencher ou se terminer
3	zones.update(vehicles, nowMs)	Recompte les véhicules dans chaque zone (4 Hz max)
4	computeFrontGaps()	Calcule la distance à la voiture de devant pour chaque véhicule
5	advanceOnRoad() × N	Déplace chaque véhicule sur sa route A*
6	snapshot()	Sérialise tous les véhicules en VehicleUpdate
7	recorder.record(frame)	Sauvegarde la frame pour le replay
8	convertAndSend()	Envoie la frame via WebSocket à tous les clients

La simulation utilise un **ReadWriteLock** : le tick tient le verrou en lecture (plusieurs threads peuvent lire), tandis que les modifications structurelles (ajout/suppression de véhicules, remplacement du graphe) tiennent le verrou en écriture. La liste de véhicules est une **CopyOnWriteArrayList** — safe pour les lectures concurrentes.

5. Les Véhicules — Types, Vitesse, Distance

Types de véhicules

Cinq types de véhicules sont simulés, chacun avec une couleur, une taille, et une vitesse de croisière différentes :

Type	Couleur (frontend)	Rôle dans la simulation
CAR	#3d9eff (bleu)	Véhicule le plus commun, vitesse moyenne
BUS	#2ecc71 (vert)	Plus lent, taille plus grande, bloque davantage
TRUCK	#e67e22 (orange)	Camion lourd, le plus lent, accidents de sévérité 3
MOTORCYCLE	#9b59b6 (violet)	Le plus rapide, petite taille
EMERGENCY	#e74c3c (rouge)	Prioritaire : jamais accidenté, immunisé aux blocages

Champs internes d'un véhicule (Vehicle.java)

Champ	Type	Description
id	String	Identifiant unique du véhicule
type	VehicleType	CAR / BUS / TRUCK / MOTORCYCLE / EMERGENCY
latitude / longitude	double	Position GPS actuelle
speed	double	Vitesse actuelle en km/h (varie selon trafic)
direction	double	Cap en degrés (0 = nord, sens horaire)
cruiseSpeed	double	Vitesse désirée sur route dégagée (km/h)
frontGap	double	Distance (m) au véhicule devant sur la même arête
wreckedUntilMs	long	Epoch ms : si > maintenant → véhicule accidenté (figé)
routeFromNode	long	ID OSM du nœud de départ de l'arête courante
routeToNode	long	ID OSM du nœud de destination de l'arête courante
routeProgress	double	Avancement sur l'arête : 0.0 (départ) → 1.0 (arrivée)
routePath	List<Long>	Séquence complète de nœuds du chemin A* courant
routePathIndex	int	Index du nœud courant dans routePath
waypointLat/Lon	double	Waypoint de secours (si pas de graphe disponible)

Calcul de position à chaque tick

La position d'un véhicule est interpolée linéairement entre les deux nœuds de l'arête courante, selon son **routeProgress** :

```
lat = fromLat + (toLat - fromLat) × progress
lon = fromLon + (toLon - fromLon) × progress
```

La **distance parcourue** par tick est calculée comme :

```
distKm = (speed_kmh / 3600.0) × dt_secondes
```

```
Et la progression sur l'arête augmente de : delta = distKm / longueur_arête_km
```

La **direction** (bearing) est calculée par la formule mathématique du bearing géodésique entre les deux nœuds OSM, en utilisant la trigonométrie sphérique.

6. Routage sur Réseau Routier Réel (A*)

Chaque véhicule ne se déplace pas en ligne droite ou aléatoirement : il suit de **vraies routes OSM** calculées par l'algorithme **A*** (A-Star), le même algorithme utilisé par les GPS.

Le graphe routier (RoadGraph.java)

Le réseau routier est représenté comme un **graphe orienté immuable** :

- **Nœuds** : intersections OSM avec leur identifiant unique (long) et leurs coordonnées GPS
- **Arêtes** : segments de route entre deux nœuds, avec leur longueur calculée par Haversine
- **Adjacence** : pour chaque nœud, la liste de ses voisins directs
- **Bounds** : la boîte englobante géographique de la carte (lat min/max, lon min/max)

Le graphe est chargé soit depuis **l'API Overpass** (internet, zone de Paris par défaut), soit depuis un **fichier .osm local** importé via l'interface. Il est **en lecture seule** une fois publié — aucun verrou n'est nécessaire pendant le tick.

Algorithme A* (AStarPathfinder.java)

L'algorithme A* trouve le chemin le plus court entre deux nœuds du graphe :

- **Heuristique** : distance Haversine jusqu'à la destination (admissible — ne surestime jamais)
- **Coût d'arête** : distance Haversine réelle entre deux nœuds voisins
- **File de priorité** : PriorityQueue binaire — complexité O((V+E) log V) en pire cas
- **Cap de sécurité** : MAX_NODES = 4 000 expansions maximum — si la destination est trop loin, le chemin partiel est retourné (évite les spikes de latence)
- **Résultat** : liste ordonnée d'IDs de nœuds [départ, n1, n2, ..., arrivée]

Stratégie de routage par véhicule

Chaque véhicule suit un chemin A* complet. Quand il arrive à destination, un nouveau chemin A* vers un nœud aléatoire est calculé (**random walk infini**). Pour éviter un spike si beaucoup de véhicules finissent en même temps :

- Un **budget de 400 calculs A* par tick** est respecté (MAX_PATHS_PER_TICK)
- Les véhicules en attente de route restent sur leur arête courante jusqu'au prochain tick
- En cas d'échec (nœud mort, graphe partiel), reassignEdge() place le véhicule sur une arête aléatoire valide

7. Modèle de Suivi de Véhicule (Car-Following)

Les véhicules ne se traversent pas. Un système de **car-following** simule le comportement réel : si un véhicule est trop proche de celui qui le précède, il ralentit. Quand un accident bloque une route, une file naturelle se forme derrière.

Calcul des distances (computeFrontGaps)

À chaque tick, tous les véhicules sont **triés** par (arête, progression) avec un seul tri O(n log n) sur un buffer réutilisé (pas d'allocation mémoire par tick). Deux véhicules successifs sur la même arête orientée sont un "suiveur" et un "leader".

```
gap = (progress_leader - progress_suiveur) × longueur_arête (en mètres)
```

Ce gap est stocké dans Vehicle.frontGap et utilisé dans advanceOnRoad().

Paramètres du modèle

Paramètre	Valeur	Signification
STOP_GAP_M	6 m	Distance à laquelle le suiveur s'arrête complètement
FOLLOW_GAP_M	45 m	Distance à partir de laquelle le suiveur commence à ralentir
ACCEL_KMH_S	9 km/h/s	Accélération maximale (≈ 2,5 m/s²)
DECEL_KMH_S	24 km/h/s	Décélération maximale (≈ 6,6 m/s²)
LIGHT_TRIGGER_M	50 m	Distance à partir de laquelle un feu rouge est pris en compte

Calcul de la vitesse cible

La vitesse cible est calculée selon le gap :

- Si gap < 6m : vitesse_cible = 0 (arrêt complet)
- Si 6m ≤ gap < 45m : vitesse_cible = cruise × (gap - 6) / (45 - 6) (proportionnel)
- Si gap ≥ 45m : vitesse_cible = cruise (vitesse de croisière libre)

La vitesse réelle n'est pas appliquée instantanément mais ajustée avec les bornes d'accélération et de décélération, ce qui crée des transitions fluides et réalistes.

8. Les Feux de Circulation

Le **TrafficLightSystem** place automatiquement des feux de signalisation sur les intersections importantes du graphe routier. Les véhicules s'arrêtent aux feux rouges et créent naturellement des files d'attente grâce au car-following.

Placement des feux

- Seuls les nœuds avec **4 voisins ou plus** (vraies intersections) reçoivent un feu
- Maximum **350 feux** par carte (pour garder le rendu et les lookups peu coûteux)
- Les candidats sont mélangés de façon déterministe (graine 0xC0FFEEL) : la même carte donne toujours les mêmes feux
- Chaque feu reçoit un **offset de cycle aléatoire** pour qu'ils ne soient pas tous synchronisés

Cycle des feux

Phase	Durée	Description
VERT	9 000 ms (9 s)	Véhicules autorisés à passer le nœud
ROUGE	7 000 ms (7 s)	Véhicules s'arrêtent à 50m avant le nœud
Cycle total	16 000 ms (16 s)	Se répète indéfiniment

L'état rouge/vert est calculé **à la volée** (pas de mutation par tick) : phase = (now + offset) mod 16000. Si phase ≥ 9000 → rouge. Aucun thread de fond, aucune allocation — pur calcul arithmétique.

Comportement des véhicules

Quand un véhicule approche d'un nœud qui a un feu, il vérifie si ce feu est rouge. Si oui, et s'il est à moins de 50m, sa **vitesse cible devient 0**. Sa progression est bloquée à 0,999 (jamais à 1,0 pour ne pas "dépasser" le feu). Dès que le feu passe au vert, le véhicule réaccélère normalement.

9. Le Système d'Accidents

L'**AccidentSystem** génère des accidents aléatoires : un véhicule est **figé sur place** pendant une durée aléatoire, devenant un obstacle statique. Les véhicules derrière font automatiquement la queue grâce au car-following — aucune logique d'embouteillage supplémentaire.

Paramètre	Valeur	Description
MAX_ACTIVE	5	Jamais plus de 5 accidents simultanés (évite le gridlock total)
Intervalle min	10 s	Délai minimum entre deux accidents
Intervalle aléatoire	+0 à 16 s	Délai supplémentaire aléatoire
Durée min d'accident	12 s	Le véhicule reste bloqué au moins 12 secondes
Durée aléatoire	+0 à 20 s	Durée supplémentaire aléatoire (max total : 32s)

- Les véhicules **EMERGENCY** sont immunisés : jamais accidentés
- Un véhicule déjà accidenté ne peut pas être accidenté à nouveau
- Le champ wreckedUntilMs stocke le timestamp de fin d'accident
- Quand un véhicule est accidenté : sa vitesse passe à 0, il ne bouge plus
- Après l'accident, le véhicule reprend son chemin A* normalement
- À chaque snapshot(), les véhicules accidentés ont wrecked=true dans le broadcast
- Le frontend affiche les véhicules accidentés avec une couleur et une taille spéciales

Impact sur le trafic

Un accident sur une arête populaire peut créer une **file de plusieurs dizaines de véhicules**. Comme chaque véhicule ralentit quand son frontGap descend sous 45m, la file se propage naturellement vers l'arrière. C'est exactement ce qui se passe dans la vraie vie.

10. Gestion du Trafic et Horloge Simulée

Le **TrafficTimeService** gère une horloge simulée qui tourne 60 fois plus vite que le temps réel. Cela permet de traverser une journée entière (heures de pointe, nuit, journée normale) en quelques minutes réelles.

Horloge simulée

- Accélération : **1 seconde réelle = 1 minute simulée** (60×)
- Démarrage à **07:00** simulé (matin, juste avant les heures de pointe)
- Cycle continu : 00:00 → 23:59 → 00:00 (journée infinie)
- Thread-safe via **AtomicLong** (accès concurrent lecture/écriture)
- Affiché dans le tableau de bord au format **HH:mm**

Modes de trafic

Mode	Heures (simulées)	Facteur de vitesse	Variance (jitter)
RUSH_HOUR	07h-09h et 17h-19h	× 0,55 (beaucoup plus lent)	± 35%
NORMAL	09h-17h et 19h-00h	× 1,0 (vitesse normale)	± 15%
NIGHT	00h-07h	× 1,35 (routes dégagées)	± 8%

Effets sur les véhicules

À chaque tick, le facteur de vitesse du mode courant est appliqué au dt effectif :

```
dt_effectif = dt × speedMultiplier × mode.speedFactor × (1 ± jitter)
```

Le **speedMultiplier** est le curseur manuel (0× à 5×) du tableau de bord. Le **mode.speedFactor** est le facteur automatique selon l'heure. Le **jitter** est la variance aléatoire par véhicule (fluidité du trafic). En RUSH_HOUR, tous les véhicules roulent à 55% de leur vitesse normale : les bouchons se forment naturellement car les gaps réduisent.

Override manuel

Le tableau de bord permet de **forcer un mode** indépendamment de l'heure simulée : POST /api/simulation/traffic/override?mode=RUSH_HOUR. Pour revenir à l'auto : POST /api/simulation/traffic/override (sans paramètre).

11. Les Zones Géographiques (Geofences)

Le **ZoneSystem** définit des **zones géographiques** polygonales sur la carte. Chaque zone compte en temps réel combien de véhicules simulés se trouvent à l'intérieur. Ce comptage est affiché dans l'overlay de zones du frontend.

Zones définies (La Rochelle)

ID	Nom	Type	Couleur
gf-zcr	Low-emission Zone	Rectangle	#2ecc71 (vert)
gf-airport	La Rochelle Airport Zone	Cercle (24 segments)	#e74c3c (rouge)
gf-delivery	Port Delivery Zone	Cercle (20 segments)	#3d9eff (bleu)
gf-school	School Safety Zone	Cercle (16 segments)	#f1c40f (jaune)

Performance du comptage

- Throttling à **4 Hz** : recalcul toutes les 250ms max (pas à chaque tick de 100ms)
- Chaque zone a une **bounding box** pré-calculée : un point hors de la bbox est rejeté en 4 comparaisons
- Pour les points dans la bbox : algorithme de **ray-casting** standard O(nb_sommets)
- Toutes les zones ont ≤ 24 sommets — test très rapide
- Pas d'allocation par tick : le buffer de comptage est réutilisé

Frontend

Le **GeofenceLayerService** affiche les zones avec trois couches MapLibre : remplissage semi-transparent (fill), contour (line), et étiquette avec le nombre de véhicules en direct (symbol). Les données viennent du endpoint GET /api/simulation/zones.

12. Le Frontend — Angular

Le frontend est une **Single Page Application Angular** (standalone components, Angular 17+). Il est organisé en modules clairement séparés : carte, véhicules, dashboard, websocket, core services.

Organisation des fichiers

Dossier / Fichier	Rôle
app.component.ts	Composant racine : shell fixe plein écran, héberge <app-map>
app.config.ts	Providers Angular : HttpClient, Animations
core/models/vehicle.model.ts	Interface VehicleUpdate, constantes VEHICLE_COLORS/SIZES
core/models/render-stats.model.ts	Interface RenderStats : fps, vehicleCount, latencyMs, renderTime
core/map-state.service.ts	Bridge signal entre MapComponent et DashboardComponent
core/map-style.service.ts	Gestion des styles de carte (CartoDB raster, OpenFreeMap vecteur)
core/map-api.service.ts	uploadMap() → POST /api/maps/upload
core/selection.service.ts	Signal selectedVehicle, FilterState, toggles
core/replay.service.ts	Machine à états du replay : idle/loading/ready/playing/paused
core/alert.service.ts	Toasts : push() / dismiss()
core/chart-data.service.ts	Historique glissant : fpsHistory, speedHistory, congestionHistory
map/map.component.ts	Hôte MapLibre, WebSocket, câblage des layers WebGL
map/heatmap.service.ts	Couche heatmap MapLibre (source + layer)
map/incident-layer.service.ts	Événements live : poll /api/simulation/events, circles MapLibre
map/geofence-layer.service.ts	Zones : fill + outline + labels MapLibre
vehicles/vehicle-renderer.ts	WebGL2 instancié, quads, interpolation, sélection
vehicles/trail-renderer.ts	WebGL2 trainées, ring buffer par véhicule
vehicles/congestion-renderer.ts	WebGL2 grille de congestion 40×30 cellules
vehicles/spatial-grid.ts	Grille spatiale uniforme, frustum culling O(k)
vehicles/vehicle-selector.ts	Clic → véhicule le plus proche dans un rayon en pixels
dashboard/dashboard.component.ts	Tous les panneaux UI, contrôles, HTTP
dashboard/search.component.ts	Filtre ID + pan-to sur la carte
dashboard/charts.component.ts	Sparklines canvas : FPS / vitesses / congestion
dashboard/alerts.component.ts	Overlay toast, auto-dismiss
dashboard/replay-bar.component.ts	Barre de replay : lecture, scrub, vitesse
dashboard/map-import.component.ts	Drag/drop .osm → upload
websocket/websocket.service.ts	STOMP over SockJS, status signal, wsLatencyMs

13. Rendu WebGL2 — Les Véhicules

Le **VehicleRenderer** est la pièce maîtresse du rendu. Il affiche tous les véhicules en un **seul appel GPU** (drawArraysInstanced) grâce au rendu instancié WebGL2. Zéro élément DOM, zéro SVG — tout passe par le GPU.

Buffer instancié (9 flottants par véhicule)

Index	Données	Description
[0]	mercatorX	Position X en coordonnées Mercator (pour la projection carte)
[1]	mercatorY	Position Y en coordonnées Mercator
[2]	r	Composante rouge de la couleur
[3]	g	Composante verte
[4]	b	Composante bleue
[5]	size	Taille du véhicule en pixels (selon le type)
[6]	directionSin	Sinus de la direction (pour la rotation)
[7]	directionCos	Cosinus de la direction
[8]	selected	1.0 si sélectionné, 0.0 sinon (anneau lumineux)

Capacité

- **20 000 slots** GPU pré-alloués (720 KB de buffer DYNAMIC_DRAW) — correspond au plafond backend
- Réassignation compacte des slots à chaque frame : les slots 0..N-1 contiennent toujours exactement les véhicules visibles
- Un seul drawArraysInstanced(count) dessine tous les véhicules visibles

Shader et formule d'invariance au zoom

Le shader vertex calcule une taille en pixels constante quelle que soit le niveau de zoom :

```
vec4 projected = u_matrix * vec4(a_position, 0.0, 1.0); vec2 rotated = rotate(a_quad, a_direction); float px = 6.0 * a_size * u_pixelRatio * pulse; vec2 offset = rotated * px * 2.0 / u_viewport * projected.w; gl_Position = projected + vec4(offset, 0.0, 0.0);
```

Le produit par .w avant addition est annulé par la division matérielle, garantissant une taille constante en pixels. L'effet pulse crée un battement animé sur le véhicule sélectionné.

Fragment shader

La distance de Manhattan est utilisée pour tracer le cercle du véhicule via smoothstep. Un anneau lumineux est ajouté pour le véhicule sélectionné. Les véhicules accidentés ont une couleur spéciale (rouge pulsant).

Interpolation de position

Le serveur envoie une frame toutes les 100ms, mais le navigateur rend à 60fps (tous les 16ms). Pour un mouvement fluide, le renderer **interpole linéairement** entre la position précédente et la position actuelle selon le temps écoulé depuis la dernière frame WebSocket.

14. Rendu WebGL2 — Trainées de Mouvement

Le **TrailRenderer** dessine une **trainée lumineuse** derrière chaque véhicule, montrant sa trajectoire récente. Activable via le bouton **T** ou le toggle "Trails" du dashboard.

- Chaque véhicule a un **ring buffer** de positions (les N dernières positions connues)
- Les trainées sont dessinées comme des lignes WebGL2 avec alpha décroissant (plus vieille = plus transparente)
- Le buffer est réutilisé et jamais réalloué — performance constante
- Un seul VAO par renderer — l'état GL de MapLibre ne peut pas corrompre les buffers
- Les trainées ont un rendu en mode '2d' : elles passent avant les bâtiments 3D

15. Rendu WebGL2 — Carte de Congestion

Le **CongestionRenderer** affiche une **grille de chaleur** de la congestion sur la carte. Chaque cellule de la grille est colorée selon la densité de véhicules et leur vitesse moyenne.

Grille 40×30 (1200 cellules)

- La zone de Paris est divisée en une grille de **40 colonnes × 30 lignes** (1200 cellules)
- Chaque cellule couvre environ 350m × 300m
- Les centres des cellules sont **pré-calculés en coordonnées Mercator** à l'initialisation
- Un seul appel drawArraysInstanced(1200) dessine toute la grille
- 5 flottants par cellule : centre X, centre Y, demi-largeur, demi-hauteur, score de congestion

Formule de score de congestion

```
score = (density / max_density) × (1 - avg_speed / 120)
```

- **density** = nombre de véhicules dans la cellule
- **avg_speed** = vitesse moyenne des véhicules dans la cellule (km/h)
- Score 0 : cellule vide ou trafic rapide → **vert**
- Score 0,5 : congestion modérée → **jaune**
- Score 1 : forte densité + vitesse faible → **rouge**

Mise à jour et rendu

- Mise à jour à **~2 Hz** (pas à chaque frame) pour réduire le coût CPU
- Le shader fragment utilise un **fondu progressif** sur les bords des cellules (smoothstep)
- Les cellules vides sont entièrement transparentes (discard dans le shader)
- Le rendu utilise le **blending alpha** pour superposer la grille sur la carte

16. Événements et Incidents en Direct

Le **IncidentLayerService** affiche sur la carte les accidents actifs en temps réel. Les données proviennent du backend (AccidentSystem) via polling REST, pas de mock — chaque point sur la carte correspond à un vrai véhicule accidenté dans la simulation.

Sources de données

Le frontend appelle GET /api/simulation/events toutes les **1,5 secondes**. Le backend renvoie la liste des véhicules dont wreckedUntilMs > now.

Champ	Valeur exemple	Description
id	acc-v-0042	ID unique de l'incident (basé sur l'ID du véhicule)
type	accident	Type d'incident (extensible : closure, construction...)
severity	1 à 3	1=moto, 2=voiture, 3=bus/camion
lat / lon	48.86, 2.35	Position GPS de l'incident
label	CAR accident	Texte affiché dans la popup
expiresAtMs	epoch ms	Timestamp de fin — utilisé pour le compte à rebours

Rendu MapLibre

- Source GeoJSON mise à jour toutes les 1,5s
- Couche circle : rayon variable selon la sévérité et le zoom
- Couleur par type : rouge (accident), orange (fermeture), jaune (congestion), bleu (travaux)
- Popup au clic : affiche le type, le label, et un **compte à rebours** jusqu'à la fin de l'incident
- Curseur pointer au survol pour indiquer la cliquabilité

17. Communication WebSocket

La communication principale entre backend et frontend utilise **STOMP over SockJS**. WebSocket est utilisé pour le flux de positions des véhicules en temps réel. Les contrôles (pause, vitesse, nb véhicules) passent par REST.

Configuration WebSocket (backend)

- Endpoint de connexion : **/ws** (SockJS fallback si WebSocket indisponible)
- Topic de publication : **/topic/vehicles**
- Broker mémoire STOMP intégré (pas de serveur externe nécessaire)
- CORS autorisé pour toutes les origines (développement)

Format du message — SimulationFrame

Chaque frame envoyée est un objet JSON :

```
{ "timestamp": 1716890000000, "vehicleCount": 500, "updates": [ { "id": "v-0001", "lat": 48.8566, "lon": 2.3522, "speed": 42.5, "direction": 135.0, "type": "CAR", "wrecked": false }, ... ] }
```

WebSocketService (frontend)

- Connexion STOMP via **SockJS** (fallback HTTP long-polling si nécessaire)
- Signal **status** : CONNECTING / CONNECTED / DISCONNECTED / ERROR
- Mesure de **wsLatencyMs** : delta entre timestamp du serveur et réception
- Reconnexion automatique en cas de coupure
- Abonnement sur /topic/vehicles → Observable

18. Tableau de Bord et Contrôles

Le **DashboardComponent** est un panneau latéral flottant contenant tous les contrôles de la simulation, les statistiques en temps réel, et les informations sur le véhicule sélectionné.

Panneaux du dashboard

Panneau	Contenu
En-tête	Logo NEXUS, badge statut WebSocket (CONNECTED/ERROR...), bouton ? (aide clavier)
Recherche	Filtre en direct par ID de véhicule + bouton pan-to (centre la carte sur le véhicule)
Stats + Trafic	4 cartes : FPS / Nb véhicules / Latence WS / Temps de rendu + horloge simulée + badge de mod
Graphiques	3 sparklines canvas : historique FPS / distribution des vitesses / congestion
Véhicule sélectionné	Indicateur pulsant + ID, type, vitesse, direction, lat/lon en temps réel
Contrôles simulation	Curseur véhicules (50–5000), curseur vitesse (0×–5×), bouton Pause/Resume
Overlays visuels	Toggles : Trainées(T) · Heatmap(H) · Congestion(C) · Mode 3D(B)
Toggles calques	Events/Incidents(I) · Zones/Geofences(G) · Cycleur de style carte
Replay	Chargement / lecture / scrub / arrêt / chips de vitesse (0,5× à 4×)
Filtres type	Chips : CAR · BUS · TRUCK · MOTORCYCLE · EMERGENCY
Aide	Référence pliable des raccourcis clavier

Raccourcis clavier

Touche	Action
H	Activer/désactiver la heatmap
T	Activer/désactiver les trainées
C	Activer/désactiver la congestion
B	Mode 3D (pitch 55°, buildings en extrusion)
I	Afficher/masquer les incidents
G	Afficher/masquer les zones géographiques
R	Cycler le mode trafic (Auto → RUSH_HOUR → NORMAL → NIGHT)
P	Pause / Reprendre la simulation
Espace	Play/Pause du replay
Échap	Désélectionner le véhicule actif

19. Filtres et Overlays

Le **SelectionService** centralise l'état des filtres et overlays. Il expose des **Signals Angular** qui déclenchent automatiquement la réactivité des composants.

FilterState

Champ	Type	Effet
types	Set<VehicleType>	Seuls les types cochés sont rendus par le VehicleRenderer
trailsEnabled	boolean	Affiche/masque le TrailRenderer
heatmapEnabled	boolean	Affiche/masque la couche heatmap MapLibre
congestionEnabled	boolean	Affiche/masque le CongestionRenderer
mode3dEnabled	boolean	Active le pitch 55° + buildings en 3D
incidentsEnabled	boolean	Affiche/masque la couche incidents
geofencesEnabled	boolean	Affiche/masque les zones géographiques

Styles de carte

Le **MapStyleService** propose deux styles de fond de carte :

- **CartoDB Raster** (défaut) : tuiles raster sombres, chargées par URL HTTP, compatible avec tous les navigateurs
- **OpenFreeMap Vecteur** : tuiles vecteur (si VECTOR_STYLE_URL est configuré), permet les bâtiments 3D

Le changement de style est géré proprement : MapLibre détruit toutes les couches custom, puis le callback style.load les réinitialise. Un guard liveSub empêche la double souscription WebSocket lors du rechargement.

20. Import de Carte OSM

Il est possible de **charger n'importe quelle zone du monde** en important un fichier .osm ou .osm.xml exporté depuis OpenStreetMap. La simulation hot-swap le graphe routier **sans redémarrage**.

Frontend — MapImportComponent

- Interface drag-and-drop ou bouton parcourir pour sélectionner un fichier .osm
- Appelle MapApiService.uploadMap(file) → POST multipart/form-data
- Affiche un indicateur de chargement, puis les stats de la carte chargée (nœuds, arêtes, bounds)

Backend — MapImportService

- Validation du type de fichier : seulement .osm ou .osm.xml
- Limite de taille : **300 MB** maximum
- Sauvegarde dans data/maps/ (persistance locale)
- Parse le XML OSM via **LocalFileRoadGraphProvider.load(path)**
- Appelle SimulationEngine.replaceGraph() sous verrou écriture
- Réinitialise l'AccidentSystem et le FrameRecorder
- Retourne : mapName, nodeCount, edgeCount, bounds

Hot-swap du graphe

Le remplacement du graphe est atomique : pendant un bref instant, la simulation est mise en pause le temps d'installer le nouveau graphe. Les véhicules sont régénérés sur les nouvelles arêtes. La pause est rétablie à son état précédent après le swap.

21. Système de Replay

Le système de **replay** permet de rejouer l'historique de la simulation. Chaque frame diffusée est enregistrée sur disque, et peut être rejouée à différentes vitesses.

Backend — FrameRecorder

- Chaque SimulationFrame est sérialisée et sauvegardée sur disque
- L'API GET /api/replay/list retourne la liste des fichiers disponibles
- L'API GET /api/replay/play/{file} stream les frames enregistrées

Frontend — ReplayService

Le **ReplayService** est une machine à états :

État	Description
idle	Aucun replay chargé
loading	Chargement du fichier replay en cours
ready	Replay prêt, pas encore en lecture
playing	Lecture en cours — les frames remplacent le flux WebSocket live
paused	Replay en pause (position maintenue)

Interface de contrôle

- Barre de scrub (timeline) pour naviguer dans le replay
- Boutons : ■■ Retour / ■ Lecture / ■ Pause / ■ Stop
- Chips de vitesse : 0,5× · 1× · 2× · 4×
- Touche Espace : play/pause depuis la carte

22. Performance et Optimisations

La performance est la **priorité n°1** du projet. Voici les choix techniques qui permettent d'afficher des milliers de véhicules sans dégradation :

Frontend — Rendu

Optimisation	Impact
WebGL2 instancié (drawArraysInstanced)	1 seul appel GPU pour N milliers de véhicules
Aucun élément DOM pour les véhicules	Zéro reflow/repaint CSS, zéro GC sur les nœuds DOM
Buffer Float32Array pré-alloué (20K slots)	Aucune allocation mémoire par frame
Isolation du change detection Angular	Les données WebSocket ne passent jamais par @Input ou Zone.js
VAO (Vertex Array Object) par renderer	L'état GL est préservé — pas de rebinding à chaque frame
Interpolation côté client (60fps)	Mouvement fluide même avec des updates serveur à 10fps
Congestion mise à jour à 2Hz	Calcul de grille coûteux pas recalculé à chaque frame
SpatialGrid pour frustum culling	Seuls les véhicules dans la vue sont traités O(k)

Backend — Simulation

Optimisation	Impact
Tick math en primitifs double	Zéro allocation par véhicule par tick
computeFrontGaps() : tri O(n log n)	Pas d'algorithme O(n^2) pour les distances
gapBuffer[] réutilisé	Aucune allocation par tick pour le tri
RoadGraph en lecture seule	Pas de verrou pendant le tick (ReadWriteLock readLock)
CopyOnWriteArrayList	Lectures concurrentes sans verrou
Budget A* : 400 max/tick	Évite les spikes lors de refontes massives de routes
Chargement graphe en thread de fond	Le serveur REST démarre immédiatement
ZoneSystem throttlé à 4Hz	Comptage de véhicules pas recalculé à chaque tick
TrafficLights : calcul à la volée	Aucune mutation par tick pour les feux

23. Structure Complète des Fichiers

Backend

`backend/src/main/java/com/nexus/`

- NexusApplication.java
- config/ SimulationConfig.java · WebSocketConfig.java · CorsConfig.java
- model/ Vehicle.java · VehicleUpdate.java · VehicleType.java · SimulationFrame.java
- model/ LightUpdate.java · IncidentUpdate.java · ZoneStatus.java · MapLoadResponse.java
- controller/ SimulationController.java · ReplayController.java · MapController.java
- service/ FrameRecorder.java · MapImportService.java · MapImportException.java
- simulation/ SimulationEngine.java · VehicleGenerator.java · RoadGraph.java
- simulation/ RoadGraphProvider.java · RoadNetworkLoader.java · LocalFileRoadGraphProvider.java
- simulation/ AStarPathfinder.java · TrafficTimeService.java · TrafficMode.java
- simulation/ AccidentSystem.java · TrafficLightSystem.java · ZoneSystem.java
- websocket/ VehicleBroadcaster.java

Frontend

`frontend/src/app/`

- app.component.ts · app.config.ts · app.routes.ts
- core/ map-state.service.ts · map-style.service.ts · map-api.service.ts
- core/ selection.service.ts · replay.service.ts · alert.service.ts · chart-data.service.ts
- core/models/ vehicle.model.ts · render-stats.model.ts
- map/ map.component.ts · heatmap.service.ts · incident-layer.service.ts · geofence-layer.service.ts
- vehicles/ vehicle-renderer.ts · trail-renderer.ts · congestion-renderer.ts
- vehicles/ spatial-grid.ts · vehicle-selector.ts
- dashboard/ dashboard.component.ts · search.component.ts · charts.component.ts
- dashboard/ alerts.component.ts · replay-bar.component.ts · map-import.component.ts
- websocket/ websocket.service.ts
- environments/ environment.ts (backendUrl, VECTOR_STYLE_URL, MAP_CENTER, MAP_ZOOM)

24. API REST — Référence Complète

Toutes les routes sont préfixées par **http://localhost:8080**

Méthode	Endpoint	Description	Réponse
GET	/api/simulation/status	État de la simulation	{vehicleCount, speedMultiplier, paused}
POST	/api/simulation/speed?multiplier=N	Modifier le multiplicateur de vitesse (0–20)	200 OK
POST	/api/simulation/vehicles?count=N	Changer le nb de véhicules (max: 5000)	200 OK
POST	/api/simulation/pause	Figer la simulation	200 OK
POST	/api/simulation/resume	Reprendre la simulation	200 OK
GET	/api/simulation/traffic	Mode trafic courant + heure simulée	{mode, simulatedTime}
POST	/api/simulation/traffic/override?mode=X	Forcer RUSH_HOUR / NORMAL / NIGHT	200 OK
POST	/api/simulation/traffic/override	Remettre en mode auto (horloge)	200 OK
GET	/api/simulation/lights	Positions + états des feux de circulation	List<LightUpdate>
GET	/api/simulation/events	Incidents actifs (accidents)	List<IncidentUpdate>
GET	/api/simulation/zones	Zones géographiques + nb véhicules	List<ZoneStatus>
GET	/api/replay/list	Fichiers replay disponibles	List<String>
GET	/api/replay/play/{file}	Stream les frames d'un replay	Stream<SimulationFrame>
POST	/api/maps/upload	Importer un fichier .osm (multipart)	MapLoadResponse

WebSocket

Topic	Payload	Fréquence
/topic/vehicles	SimulationFrame {timestamp, vehicleCount, List<VehicleUpdate>}	Toutes les 100ms

Configuration (application.yml)

```
nexus: simulation: vehicle-count: 500 # nb initial de véhicules update-rate-ms: 100 # intervalle broadcast WebSocket max-vehicles: 5000 # plafond absolu graph-source: overpass # 'overpass' ou 'local-file' local-file: data/maps/map.osm area: lat-min: 48.82 lat-max: 48.90 lon-min: 2.28 lon-max: 2.42
```

Conclusion

NexusNexus est une application de visualisation trafic full-stack qui démontre la maîtrise de plusieurs domaines techniques :

- **Architecture distribuée** : frontend et backend clairement séparés, communicant via WebSocket et REST
- **Algorithmes** : A* de Dijkstra, car-following, ray-casting, Haversine, interpolation
- **Rendu GPU** : WebGL2 instancié, shaders GLSL, VAO, ring buffers
- **Temps réel** : WebSocket STOMP, simulation à 10fps, rendu à 60fps avec interpolation
- **Performance** : aucune allocation par tick, O(n log n) pour les distances, budget A*
- **Géographie** : graphe routier OSM réel, projection Mercator, coordonnées GPS
- **Ingénierie logicielle** : architecture modulaire, faible couplage, code maintenable

L'application peut être démarrée avec deux commandes :

```
# Backend cd backend && ./mvnw spring-boot:run # Frontend cd frontend && npm install && npm start
```

Puis ouvrir **http://localhost:4200** pour accéder à l'application.